Phishing URL Detection System Using Machine Learning Techniques

PROJECT REPORT

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING

By
Batch-6

Jami Siri (323103210085)

Jasti Taruni (322103210087)

Javvadi Swathi (323103210088)

J. Leela Bhavani (323103210089)

Under the esteemed guidance of
Dr. N. Sharmili
Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B.Tech.CSE,ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi,Madhurawada,Visakhapatnam–530048

2023 –2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled "**Phishing URL Detection System Using Machine Learning Techniques**" is the bonafide work of following III B.Tech. students from the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women, affiliated with AndhraUniversity, Visakhapatnam. This project was done **as part of Cryptography and Network Security lab** during the academic year 2025–2026 (III Year Semester-2).

.

Jami Siri (323103210085)
Jasti Taruni (322103210087)
Javvadi Swathi (323103210088)
J. Leela Bhavani (323103210089)

Signature of the Guide

Dr. N. Sharmili MTech.,Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

ABSTRACT

This project presents a **Phishing URL Detection System designed to identify and prevent malicious websites** that attempt to steal sensitive user information. The main objective of this system is to enhance online security by analyzing URLs using rule-based heuristic techniques to classify them as legitimate or phishing.

In this system, the user provides a URL as input, which is then analyzed based on predefined rules and patterns commonly associated with phishing attacks. These rules include checking factors such as URL length, presence of special characters (like '@' or '-'), use of IP addresses instead of domain names, suspicious keywords, multiple subdomains, and absence of secure HTTPS protocols.

The system evaluates these features and assigns a decision based on how many suspicious characteristics are detected. The backend is implemented using Flask, where the logic for rule-based analysis is executed, and the result is displayed through a user-friendly interface.

If the URL passes the defined safety checks, it is classified as legitimate; otherwise, it is flagged as a phishing URL. This approach provides a fast and efficient method for detecting potentially harmful links without requiring complex machine learning models.

This solution helps users identify unsafe websites, reduces the risk of phishing attacks, and promotes safer browsing practices by offering a simple and effective real-time URL verification system.

1.SYSTEM REQUIREMENTS

1.1. Software Requirements

Table 1.Software Requirements

Software	Version/ Specification
Python	3.10.12(64-bit)
Visual Studio Code Editor	1.85+
Operating System	Windows 11
Flask	2.x
NumPy	1.24.4
HTML	Standard
CSS	Standard
Streamlit	1.30+
Scikit-learn	1.3+

Python packages

Python packages we used in our project includes the following table 2.

We used the command pip install "*package_name==version*" to download the packages.

Table 2.Python packages

S.No.	PackageUsed	Description
1.	NumPy	Used for numerical computations and handling multi-dimensional arrays efficiently.
2.	Pandas	Used for data manipulation and analysis using DataFrames and Series. series
3.	Scikit-learn	Provides tools for machine learning tasks like classification and regression.
4.	Pickle	Used to save and load the trained machine learning model.
5.	Regex (re)	Used for pattern matching and text processing using regular expressions.

FrontEndPackagesused

Table 3.FrontendPackages

S.NO	LibrariesUsed	Description
1.	Stream lit	Used to build interactive web applications for data visualization and machine learning with minimal code.
2.	Jinja2	Used for rendering dynamic HTML templates in web applications like Flask.
3.	Virtualenv	Used to create isolated Python environments for managing dependencies separately.
4.	OS	Provides the environment to run applications and manage hardware and system resources.

1.2. Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

Minimum Hardware Requirements:

Table 4. Minimum Hardware Requirements

S.No	Hardware Component	Specification
1.	Storage Device	Minimum 100 GB HDD/SSD for storing files, QR codes, and keys
2.	Processor	64-bit processor (Intel/AMD) for efficient execution of Python and cryptographic operations
3.	RAM	Minimum 4 GB
4.	Internet Connection	Required for downloading libraries, updates, and deployment
5.	Display Unit	Monitor with at least 1366×768 resolution
6.	Input Devices	Keyboard and Mouse for user interaction

2.METHODOLOGY

2.1 SYSTEM ARCHITECTURE

The proposed Phishing URL Detection System follows a client-server architecture implemented using the Streamlit framework. The system is designed as a user-interactive application where the frontend and backend are integrated into a single platform for simplicity and efficiency. The system primarily consists of two main components: the Feature Extraction Module and the Detection Module, supported by machine learning processing and rule-based heuristic analysis.

The Feature Extraction Module processes the input URL provided by the user and extracts various characteristics such as URL length, presence of IP address, special symbols, suspicious keywords, and structural patterns. These extracted features are then passed to the Detection Module.

The Detection Module utilizes a trained machine learning model along with rule-based conditions to classify the URL as either legitimate or phishing. The system combines both approaches to improve accuracy and handle edge cases effectively. Based on the analysis, the result is displayed to the user through an intuitive interface.

This architecture ensures fast processing, real-time detection, and ease of use, making it an efficient solution for identifying potentially harmful URLs and enhancing online security.

2.2 DESCRPTION, ALGORITHM

System Description

The Phishing URL Detection System is designed to identify and classify malicious URLs that may lead to phishing attacks. The system uses machine learning techniques along with rule-based heuristic analysis to ensure accurate and efficient detection. The system is implemented using the Streamlit framework for frontend interaction and Python-based processing for backend operations.

The system operates in two main phases:

1. Feature Extraction Phase

In this phase, the system processes the URL entered by the user and extracts relevant features required for analysis. The process is as follows:

• The user inputs a URL through the application interface.
• The system analyzes the URL structure and extracts multiple features such as:
o URL length
o Presence of IP address instead of domain name
o Use of special symbols like '@' and '-'
o Number of dots, slashes, and hyphens
o Presence of suspicious keywords like *login, verify, bank, secure, account, update*
o Detection of HTTPS usage
o Identification of fake HTTPS patterns
o Presence of numeric characters in the URL
• These extracted features are converted into a structured numerical format.
• The feature vector is then passed to the detection module for classification.

2. Detection and Classification Phase

In this phase, the system determines whether the given URL is legitimate or phishing:

• The extracted feature vector is provided as input to the trained machine learning model (Logistic Regression).
• The model processes the input and predicts whether the URL is phishing or legitimate.
• The system also calculates a confidence score using probability estimation from the model.
• In addition to machine learning, rule-based conditions are applied for critical checks such as:
o Presence of IP address in the URL
o Usage of '@' symbol indicating redirection
• Based on both ML prediction and rule-based validation, the system classifies the URL as:
o Legitimate URL
o Phishing URL
• The final result along with confidence percentage is displayed to the user through the interface.

Final Decision:

• If the URL passes all checks and is predicted safe → it is classified as **Legitimate**.

• If suspicious patterns or phishing indicators are detected → it is classified as **Phishing**.

Algorithm:

Step 1: Model Preparation

1. Collect dataset containing URL features and labels (legitimate/phishing).
2. Define input features such as URL length, presence of IP address, special characters, keywords, etc.
3. Load dataset using Pandas.
4. Separate features (X) and target labels (y).
5. Train a Logistic Regression model using Scikit-learn.
6. Save the trained model as model.pkl using Pickle.
7. If model already exists, reuse it without retraining.

Step 2: Feature Extraction Process

1. User enters a URL in the application interface.
2. The system receives the input URL.
3. Extract features from the URL including:
   o URL length
   o Presence of IP address
   o Presence of '@' symbol
   o Use of HTTPS protocol
   o Suspicious keywords (login, verify, bank, secure, etc.)
   o Count of dots (.)
   o Count of slashes (/)
   o Count of hyphens (-)
   o Domain length
   o Detection of fake HTTPS usage
   o Presence of numeric characters
4. Convert extracted features into numerical format.
5. Form a feature vector for model input.

Step 3: Detection Process

1. Load the trained ML model from model.pkl.
2. Pass the extracted feature vector to the model.
3. Predict whether the URL is phishing or legitimate.
4. Calculate prediction probability (confidence score).
5. Apply rule-based checks for critical conditions:
   o If URL contains IP address
   o If URL contains '@' symbol
6. Combine ML prediction with rule-based logic for final decision.

Step 4: Result Decision

1. If rule-based conditions detect strong phishing indicators →
   Display **"Phishing URL Detected"**
2. Else if ML model predicts phishing →
   Display **"Phishing URL Detected"**
3. Else →
   Display **"Legitimate URL"**
4. Display confidence percentage of prediction.
5. (Optional) If confidence is moderate →
   Display **"Suspicious URL"**

3. IMPLEMENTATION

Source code:

Model.py

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
import pickle

# Load dataset
data = pd.read_csv("phishing.csv")

X = data.drop("label", axis=1)
y = data["label"]

# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X, y)

# Save model
pickle.dump(model, open("model.pkl", "wb"))

print("Model trained successfully!")
```

App.py

```
import streamlit as st
import re
import pickle

# Load model
model = pickle.load(open("model.pkl", "rb"))

st.title("□ Phishing URL Detector (ML + Rule-Based)")

url = st.text_input("Enter URL")
```

```
def extract_features(url):
    url_length = len(url)

    has_ip = 1 if re.search(r"\d+\.\d+\.\d+\.\d+", url) else 0
    has_at = 1 if '@' in url else 0
    has_https = 1 if url.startswith("https") else 0

    keywords = ['login', 'verify', 'bank', 'secure', 'account', 'update']
    has_keywords = 1 if any(word in url.lower() for word in keywords) else 0

    dot_count = url.count('.')
    slash_count = url.count('/')
    dash_count = url.count('-')

    domain_length = len(url.split("//")[-1])

    fake_https = 1 if "https" in url and not url.startswith("https") else 0

    has_numbers = 1 if any(char.isdigit() for char in url) else 0

    return [[
        url_length,
        has_ip,
        has_at,
        has_https,
        has_keywords,
        dot_count,
        slash_count,
        dash_count,
        domain_length,
        fake_https,
        has_numbers
    ]], has_ip, has_at
```

```
if st.button("Check URL"):
    if url:
        features, has_ip, has_at = extract_features(url)

        prediction = model.predict(features)[0]
        probs = model.predict_proba(features)[0]

        if prediction == 1:
            prob = probs[1]
        else:
            prob = probs[0]

        # Hybrid rule override
        if prediction == 1 or has_ip or has_at:
            st.error("❌Phishing URL Detected")
        else:
            st.success("✅Legitimate URL")

        st.write(f"Confidence: {prob*100:.2f}%")

        if 40 < prob*100 < 60:
            st.warning("⚠ Suspicious URL")

    else:
        st.warning("Please enter a URL")
```

phishing.csv:

```
url_length,has_ip,has_at,has_https,has_keywords,dot_count,slash_count,dash_count,domain_
length,fake_https,has_numbers,label
25,0,0,1,0,2,2,0,20,0,0,0
30,0,0,1,0,2,2,0,25,0,0,0
35,0,0,1,0,2,2,0,30,0,0,0
```

40,0,0,1,0,3,3,0,35,0,0,0
45,0,0,1,0,3,3,0,40,0,0,0
50,0,0,1,0,3,3,0,45,0,0,0
55,0,0,1,0,3,3,0,50,0,0,0
60,0,0,1,0,3,3,0,55,0,0,0
65,0,0,1,0,3,3,0,60,0,0,0
70,0,0,1,0,3,3,0,65,0,0,0
75,0,0,1,0,3,3,0,70,0,0,0
80,0,0,1,0,4,3,0,75,0,0,0
85,0,0,1,1,4,3,0,80,0,0,0
90,0,0,1,1,4,3,0,85,0,0,0
95,0,0,1,1,4,3,0,90,0,0,0
100,0,0,1,1,4,3,0,95,0,0,0

110,1,0,0,1,4,4,2,100,0,1,1
115,1,0,0,1,4,4,2,105,0,1,1
120,1,0,0,1,4,4,2,110,0,1,1
125,1,0,0,1,5,4,2,115,0,1,1
130,1,0,0,1,5,4,2,120,0,1,1
135,1,0,0,1,5,5,3,125,0,1,1
140,1,1,0,1,5,5,3,130,0,1,1
145,1,1,0,1,5,5,3,135,0,1,1
150,1,1,0,1,6,5,3,140,0,1,1
155,1,1,0,1,6,5,3,145,0,1,1
160,1,1,0,1,6,6,4,150,0,1,1
165,1,1,0,1,6,6,4,155,0,1,1
170,1,1,0,1,6,6,4,160,0,1,1
175,1,1,0,1,6,6,4,165,0,1,1
180,1,1,0,1,7,6,4,170,0,1,1
185,1,1,0,1,7,6,4,175,0,1,1
190,1,1,0,1,7,7,5,180,0,1,1
195,1,1,0,1,7,7,5,185,0,1,1
200,1,1,0,1,7,7,5,190,0,1,1
210,1,1,0,1,8,7,5,200,0,1,1

95,0,1,0,1,4,3,2,90,0,1,1

100,0,1,0,1,4,3,2,95,0,1,1

105,0,1,0,1,4,3,2,100,0,1,1

110,0,1,0,1,5,4,2,105,0,1,1

115,0,1,0,1,5,4,2,110,0,1,1

120,0,1,0,1,5,4,3,115,0,1,1

125,0,1,0,1,5,4,3,120,0,1,1

130,0,1,0,1,6,5,3,125,0,1,1

135,0,1,0,1,6,5,3,130,0,1,1

140,0,1,0,1,6,5,3,135,0,1,1

Style.css:

```
body {
    background-color: #0f172a;
    font-family: Arial, sans-serif;
    color: white;
}

.container {
    width: 400px;
    margin: 100px auto;
    padding: 20px;
    background-color: #1e293b;
    border-radius: 10px;
    text-align: center;
}
input {
    width: 90%;
    padding: 10px;
    margin: 10px 0;
    border-radius: 5px;
    border: none;
}
```

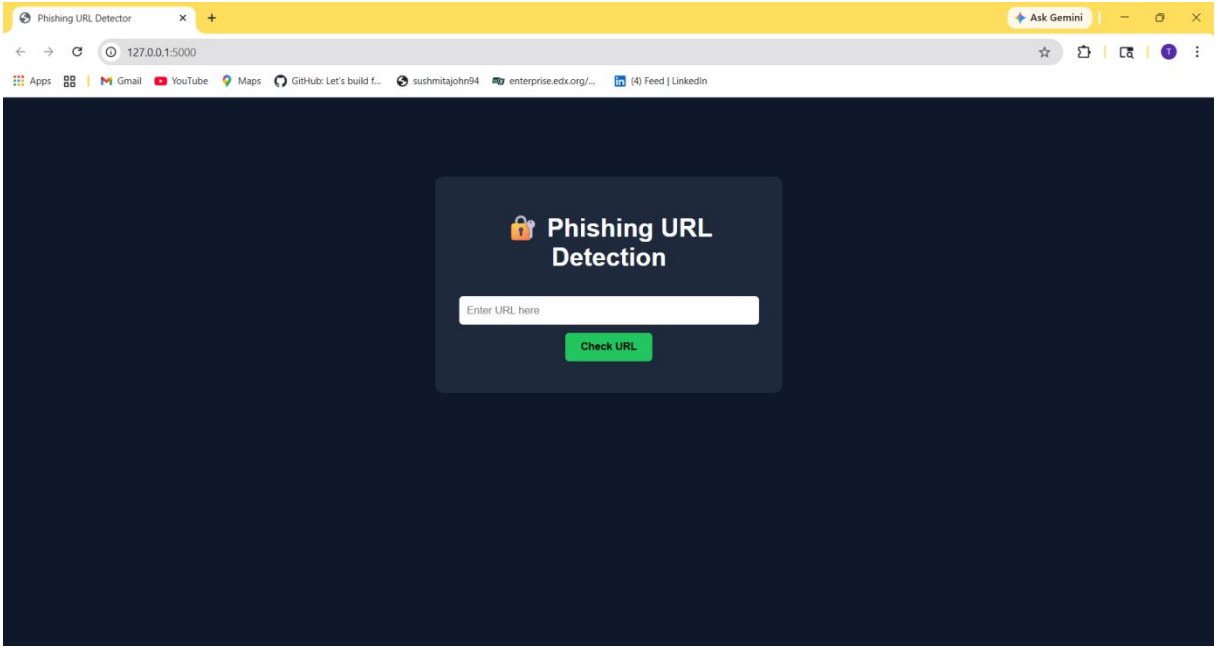
```
button {
    padding: 10px 20px;
    background-color: #22c55e;
    border: none;
    font-weight: bold;
    cursor: pointer;
    border-radius: 5px;
}
h2 {
    margin-top: 20px;
}
```

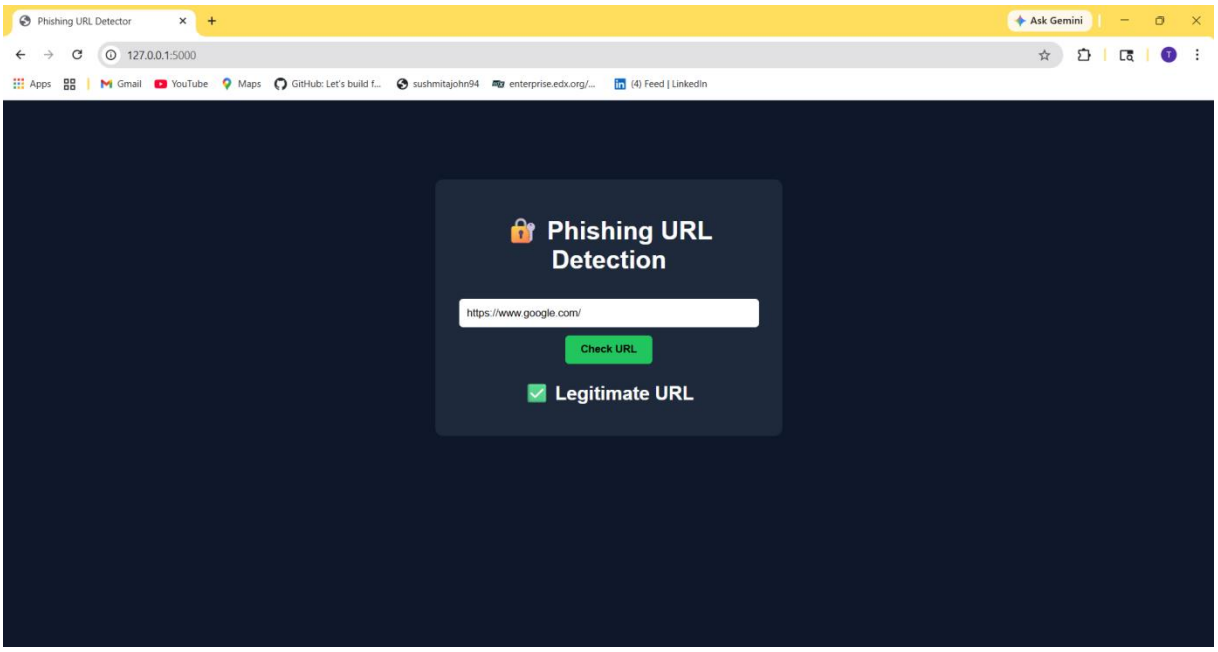
Index.html:

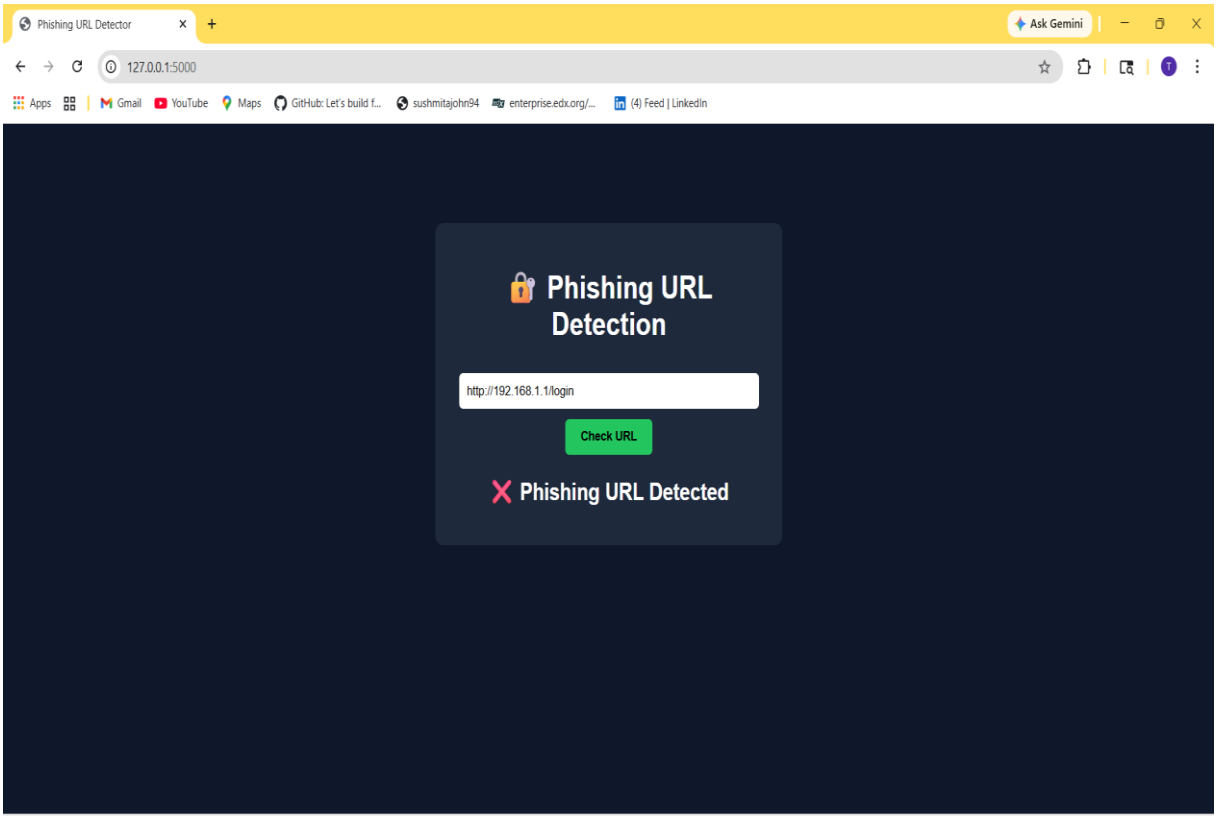
```
<!DOCTYPE html>
<html>
<head>
    <title>Phishing URL Detector</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="container">
    <h1>□ Phishing URL Detection</h1>

    <form method="POST">
        <input type="text" name="url" placeholder="Enter URL here" value="{{ url }}"
required>
        <button type="submit">Check URL</button>
    </form>
    <h2>{{ result }}</h2>
</div>

</body>
</html>
```

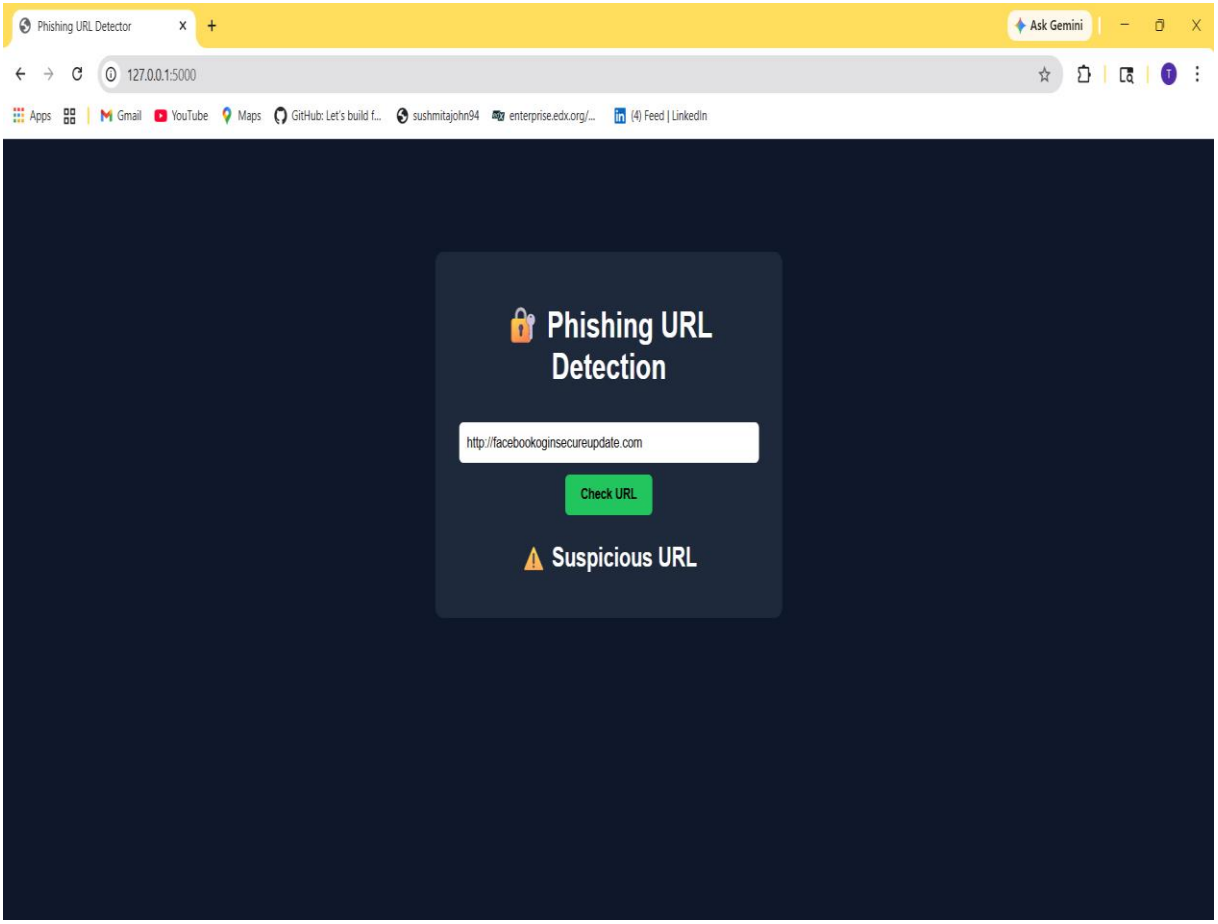
4. RESULTS

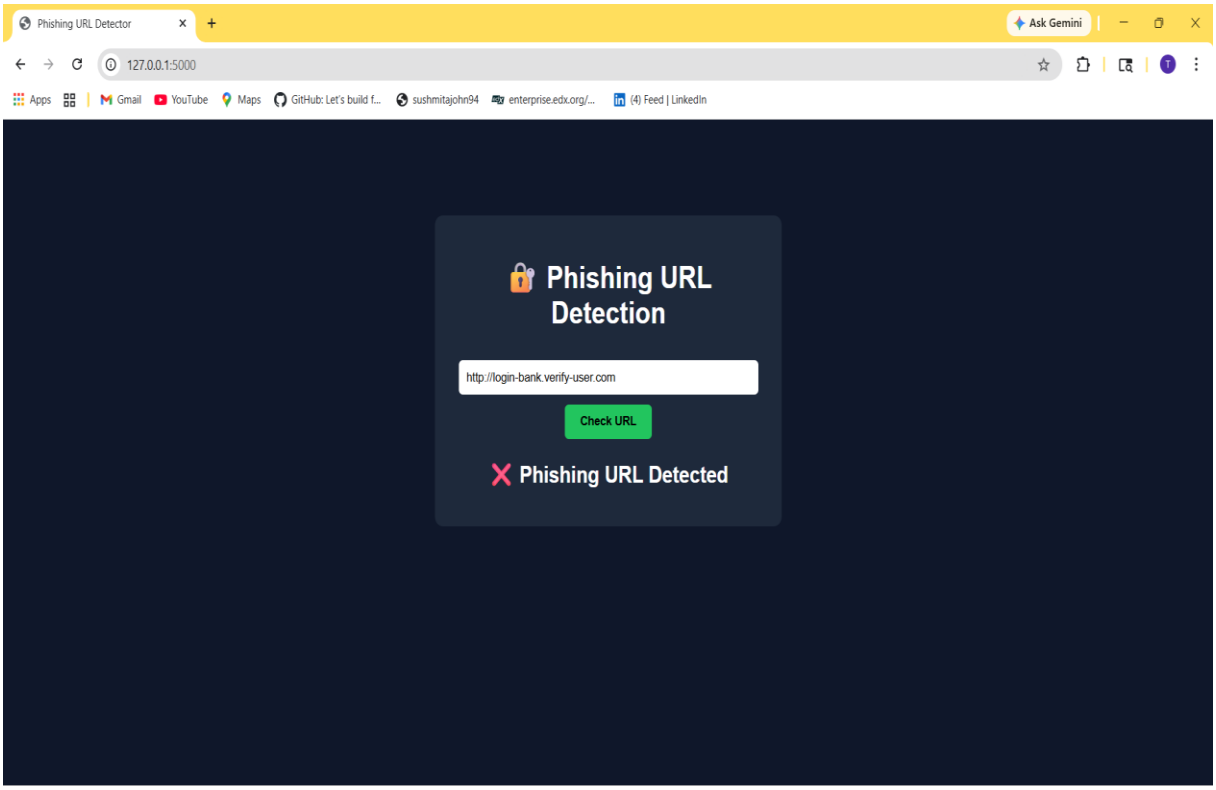
4.1 OutputScreens:

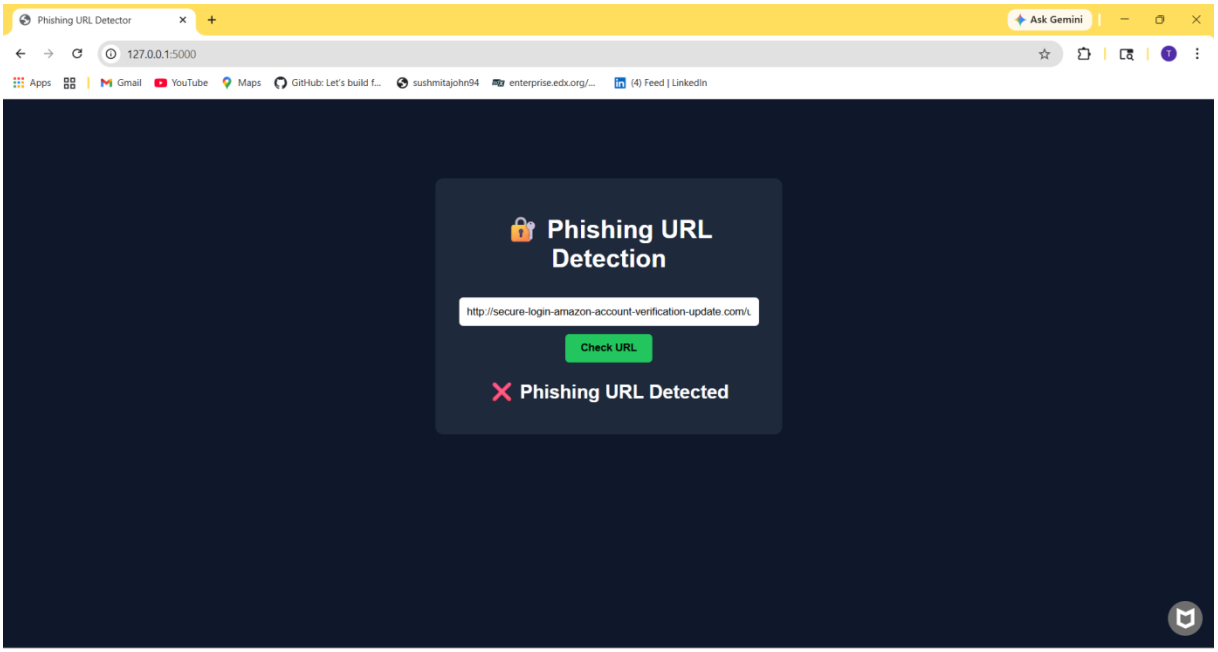





Phishing URL Detector
Ask Gemini
127.0.0.1:5000
Apps
Gmail
YouTube
Maps
GitHub: Let's build f...
sushmitajohn94
enterprise.edx.org/...
(4) Feed | LinkedIn
Phishing URL Detection
http://192.168.1.1/login
Check URL
Phishing URL Detected


Phishing URL Detector
Ask Gemini
127.0.0.1:5000
Apps
Gmail
YouTube
Maps
GitHub: Let's build f...
sushmitajohn94
enterprise.edx.org/...
(4) Feed | LinkedIn
Phishing URL Detection
http://facebookoginsecureupdate.com
Check URL
Suspicious URL


Phishing URL Detector
Ask Gemini
127.0.0.1:5000
Apps
Gmail
YouTube
Maps
GitHub: Let's build f...
sushmitajohn94
enterprise.edx.org/...
(4) Feed | LinkedIn
Phishing URL Detection
http://login-bank.verify-user.com
Check URL
Phishing URL Detected


Phishing URL Detector
Ask Gemini
127.0.0.1:5000
Apps
Gmail
YouTube
Maps
GitHub: Let's build f...
sushmitajohn94
enterprise.edx.org/...
(4) Feed | LinkedIn
Phishing URL Detection
http://secure-login-amazon-account-verification-update.com/
Check URL
Phishing URL Detected

5. CONCLUSION

The Phishing URL Detection System successfully provides a reliable solution for identifying malicious and fraudulent websites. The system integrates machine learning techniques along with rule-based heuristic analysis to ensure effective detection of phishing URLs. By analyzing various features such as URL structure, presence of suspicious patterns, keywords, and security indicators, the system is able to classify URLs accurately as legitimate or phishing.

The use of Logistic Regression enables efficient prediction based on extracted features, while rule-based conditions further enhance the system's ability to detect obvious phishing attempts. The Streamlit framework offers a simple and interactive user interface, allowing users to quickly verify URLs in real time.

The implementation reduces the risk of phishing attacks, promotes safer browsing practices, and provides users with immediate feedback on potentially harmful links. It also minimizes dependency on manual inspection and improves overall cybersecurity awareness.

Overall, this project demonstrates how machine learning combined with heuristic techniques can be effectively used to detect phishing attacks and enhance online security in real-world applications such as online banking, e-commerce, and digital communication platforms.

.

6. FUTURE SCOPE

Although the current Phishing URL Detection System is functional and effective, it can be further enhanced with the following improvements:

1. Large-Scale Dataset Integration

• Use real-world phishing datasets with thousands of URLs to improve model accuracy.

• Enables better generalization and detection of complex phishing patterns.

2. Advanced Machine Learning Models

• Implement advanced algorithms such as Random Forest, Support Vector Machines (SVM), or Neural Networks.

• Improves prediction accuracy and handles complex URL patterns more effectively.

3. Real-Time Threat Intelligence Integration

• Integrate APIs like Google Safe Browsing or VirusTotal.

• Allows real-time verification of URLs using global threat databases.

4. Browser Extension Development

• Develop a browser extension for Chrome/Edge to detect phishing links instantly.

• Provides real-time protection while browsing.

5. Mobile Application

• Build an Android/iOS application for checking URLs on mobile devices.

• Improves accessibility and user convenience.

6. Enhanced Feature Engineering

• Include additional features such as domain age, WHOIS information, DNS records, and SSL certificate validation.

• Improves detection of sophisticated phishing attacks.

7. Cloud Deployment

• Deploy the system on cloud platforms like AWS, Azure, or Render.

• Enables global access and scalability.

8. Multi-Level Classification System

• Introduce more categories such as Legitimate, Suspicious, and Phishing.

• Provides more detailed analysis for users.

9. Continuous Learning System

• Implement model retraining using new data collected over time.

• Keeps the system updated with evolving phishing techniques.

7. REFERENCES

- William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education.
- Flask Official Documentation :https://flask.palletsprojects.com/
- OpenCV Documentation :https://opencv.org/
- Python Cryptography Library Documentation: https://cryptography.io/
- QR Code Standard Information: https://www.qrcode.com/
- National Institute of Standards and Technology (NIST), *Secure Hash Standard (SHA-256)*.
- Rivest, Shamir, and Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," Communications of the ACM.
- qrcode Python library Documentation: https://pypi.org/project/qrcode/